

Reinforcement Learning 1

Anders Lyhne Christensen

Supervised Learning

Regression

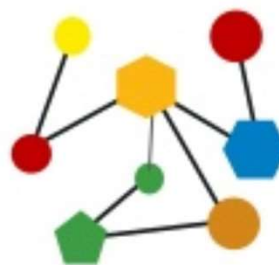


Classification

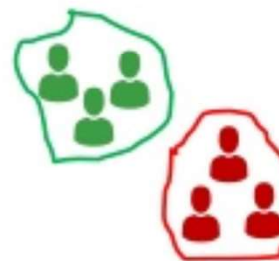


Unsupervised Learning

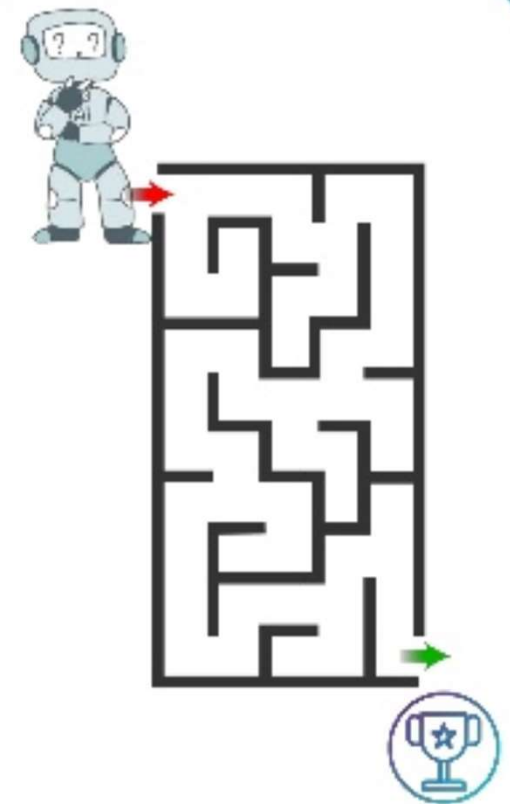
Association



Clustering



Reinforcement Learning



Supervised Learning

Map labelled input to known output

Labelled Input

Training

Algorithm

Known Output

Unsupervised Learning

Understand patterns and discover output

Unlabelled Input

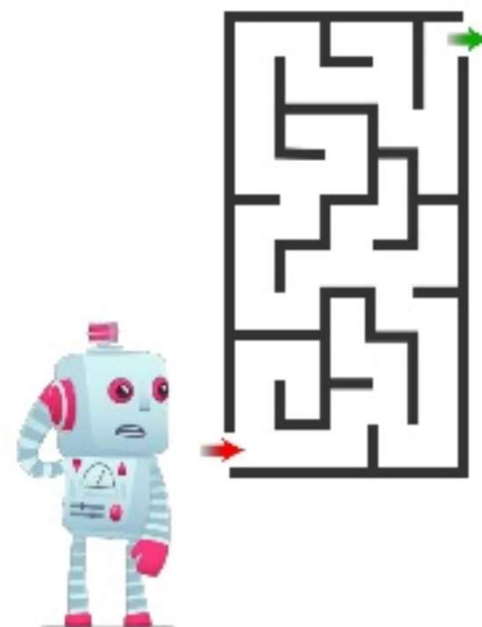
Explore patterns & trends

Algorithm

Output

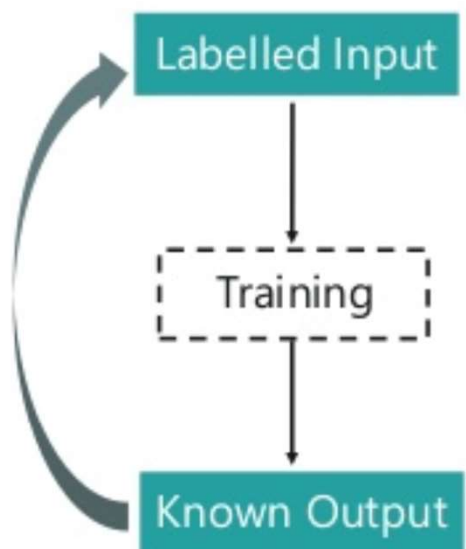
Reinforcement Learning

Follow Trail and Error method



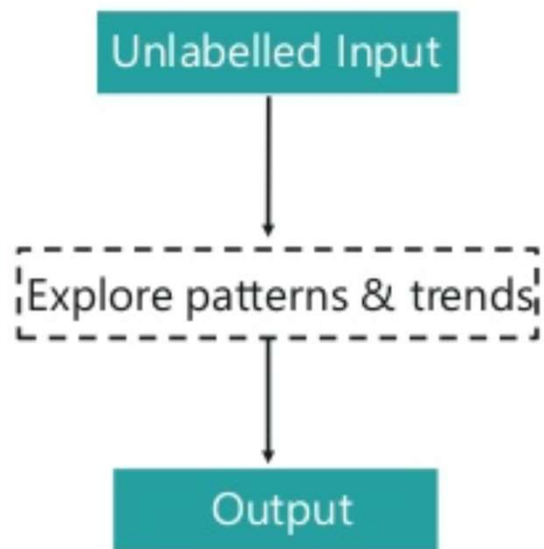
Supervised Learning

Direct Feedback



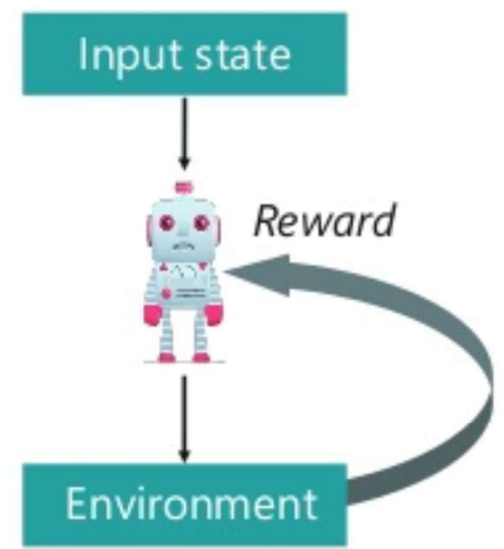
Unsupervised Learning

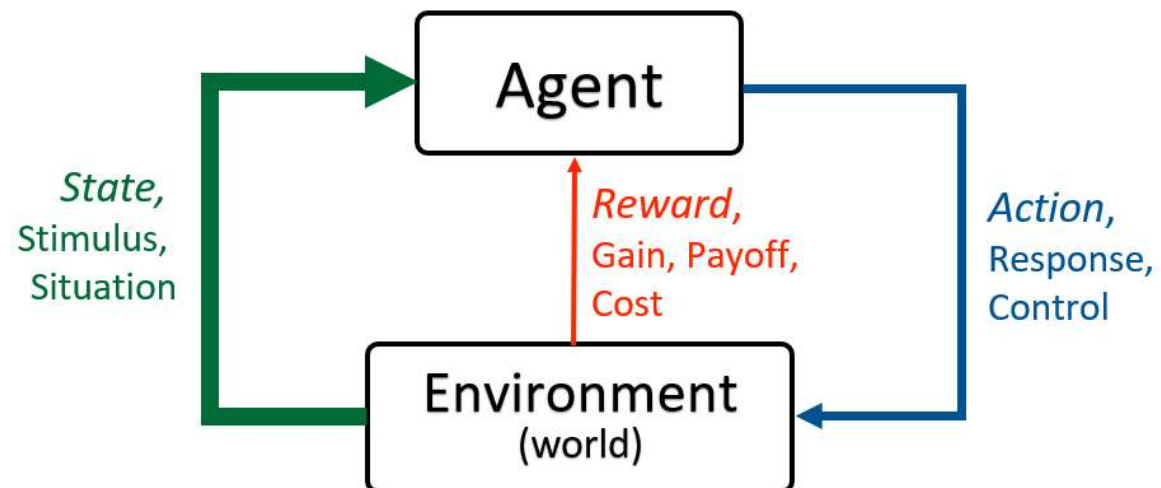
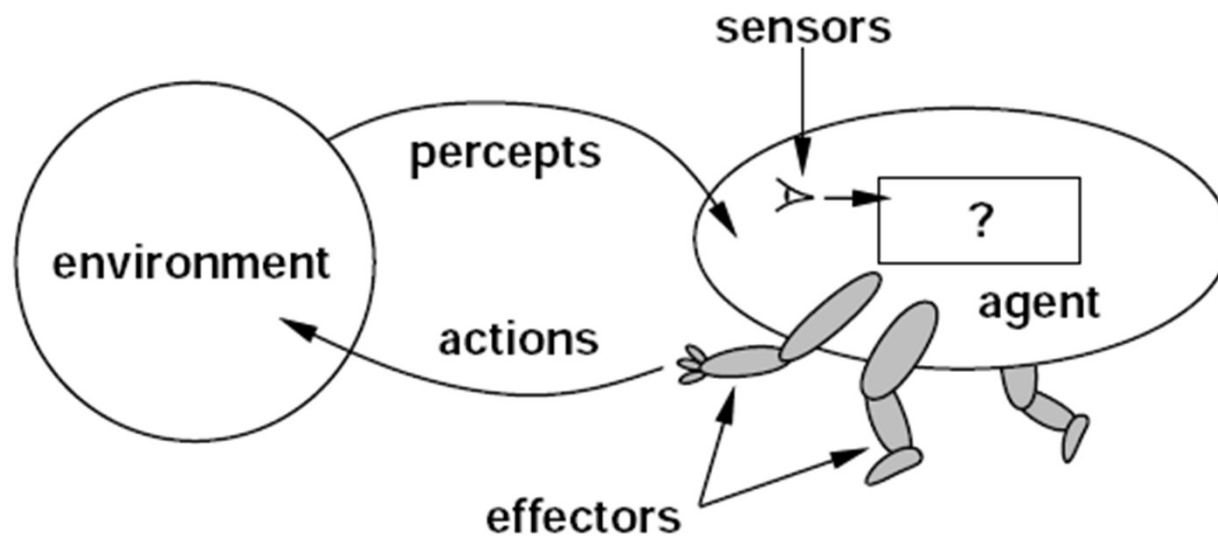
No Feedback



Reinforcement Learning

Reward system





What is the reward?



Challenges

- An agent that typically has **no prior knowledge** must learn how to solve a task through interaction with an environment.
- The agent may receive a reward after each action, or only from time to time – perhaps only very infrequently.



Example: Learning to play football from scratch is challenging: when should an agent be rewarded?

Sparse and deceptive rewards can complicate learning.

Multiarmed/ k -armed Bandit Problem

Multiarmed/ k -armed Bandit Problem

Suppose there are k bandits.

Action a_i is like pulling a slot machine arm with random payoff function $R(a_i)$



$R(a_1)$



$R(a_2)$

...



...

$R(a_k)$

Definitions

$$q_*(a) \doteq \mathbb{E}[R_t \mid A_t = a]$$

Expected reward

Arbitrary action

Expected value

Reward at time t

Action selected at time t

The diagram illustrates the definition of the optimal action-value function $q_*(a)$. The equation is $q_*(a) \doteq \mathbb{E}[R_t \mid A_t = a]$. Handwritten orange annotations provide context for each part of the equation:

- Expected reward**: Points to q_* .
- Arbitrary action**: Points to a .
- Expected value**: Points to $\mathbb{E}$.
- Reward at time t** : Points to R_t .
- Action selected at time t** : Points to A_t .

Assume that there are $k=4$ bandits

Assume that there are $k=4$ bandits

- Action: pull arm 1 — Reward is always 8
 - value of arm 1 is $q_*(1) =$

Assume that there are $k=4$ bandits

- Action: pull arm 1 — Reward is always 8
 - value of arm 1 is $q_*(1) = 8$

Assume that there are $k=4$ bandits

- Action: pull arm 1 — Reward is always 8
 - value of arm 1 is $q_*(1) = 8$
- Action: pull arm 2 — 88% chance of 0 and 12% chance of 100!
 - value of arm 2 is $q_*(2) = .88 \times 0 + .12 \times 100 =$

Assume that there are $k=4$ bandits

- Action: pull arm 1 — Reward is always 8
 - value of arm 1 is $q_*(1) = 8$
- Action: pull arm 2 — 88% chance of 0 and 12% chance of 100!
 - value of arm 2 is $q_*(2) = .88 \times 0 + .12 \times 100 = 12$

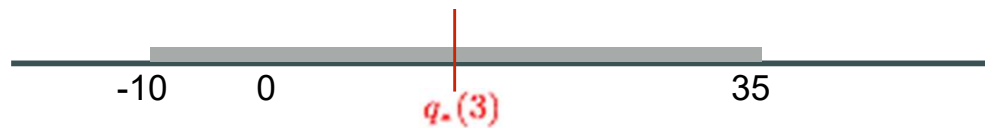
Assume that there are $k=4$ bandits

- Action: pull arm 1 — Reward is always 8
 - value of arm 1 is $q_*(1) = 8$
- Action: pull arm 2 — 88% chance of 0 and 12% chance of 100!
 - value of arm 2 is $q_*(2) = .88 \times 0 + .12 \times 100 = 12$
- Action: pull arm 3 — Randomly between -10 and 35, equiprobable



Assume that there are $k=4$ bandits

- Action: pull arm 1 — Reward is always 8
 - value of arm 1 is $q_*(1) = 8$
- Action: pull arm 2 — 88% chance of 0 and 12% chance of 100!
 - value of arm 2 is $q_*(2) = .88 \times 0 + .12 \times 100 = 12$
- Action: pull arm 3 — Randomly between -10 and 35, equiprobable



$$q_*(3) = 12.5$$

Assume that there are $k=4$ bandits

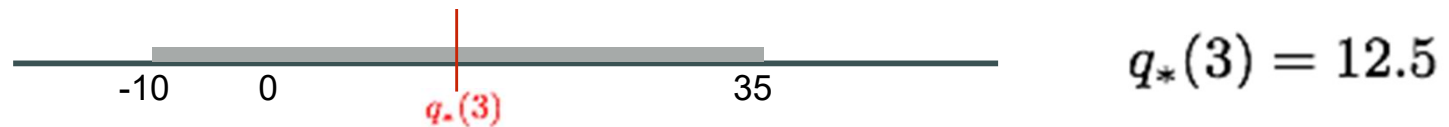
- Action: pull arm 1 — Reward is always 8

- value of arm 1 is $q_*(1) = 8$

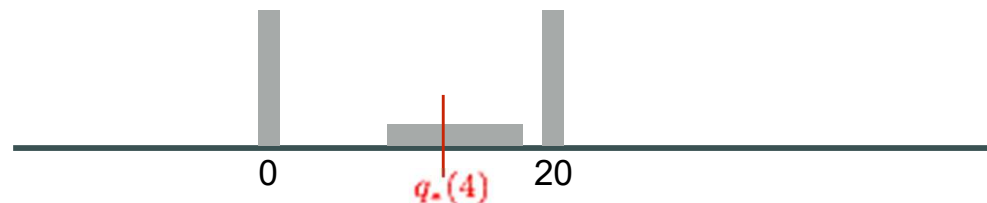
- Action: pull arm 2 — 88% chance of 0 and 12% chance of 100!

- value of arm 2 is $q_*(2) = .88 \times 0 + .12 \times 100 = 12$

- Action: pull arm 3 — Randomly between -10 and 35, equiprobable



- Action: pull arm 4 — a third 0, a third 20, a third from $\{8, 9, \dots, 18\}$



$q_*(4) =$

Assume that there are $k=4$ bandits

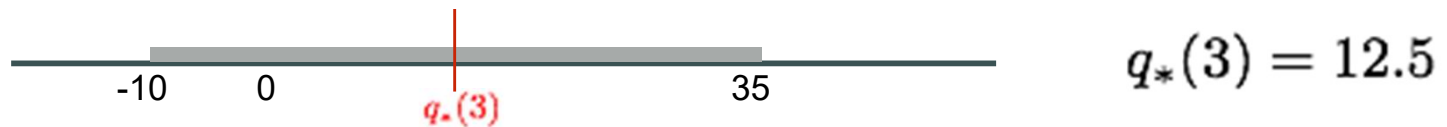
- Action: pull arm 1 — Reward is always 8

- value of arm 1 is $q_*(1) = 8$

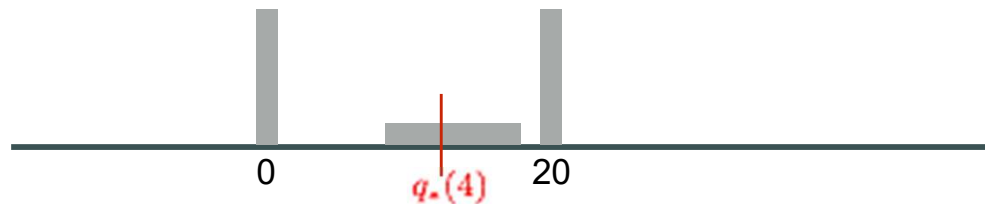
- Action: pull arm 2 — 88% chance of 0 and 12% chance of 100!

- value of arm 2 is $q_*(2) = .88 \times 0 + .12 \times 100 = 12$

- Action: pull arm 3 — Randomly between -10 and 35, equiprobable



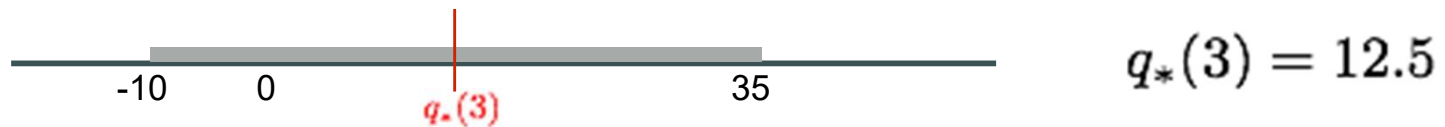
- Action: pull arm 4 — a third 0, a third 20, a third from $\{8,9,\dots, 18\}$



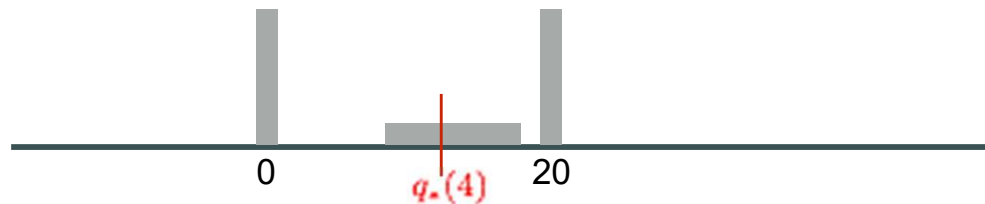
$$q_*(4) = \frac{1}{3} \times 0 + \frac{1}{3} \times 20 + \frac{1}{3} \times$$

Assume that there are $k=4$ bandits

- Action: pull arm 1 — Reward is always 8
 - value of arm 1 is $q_*(1) = 8$
- Action: pull arm 2 — 88% chance of 0 and 12% chance of 100!
 - value of arm 2 is $q_*(2) = .88 \times 0 + .12 \times 100 = 12$
- Action: pull arm 3 — Randomly between -10 and 35, equiprobable



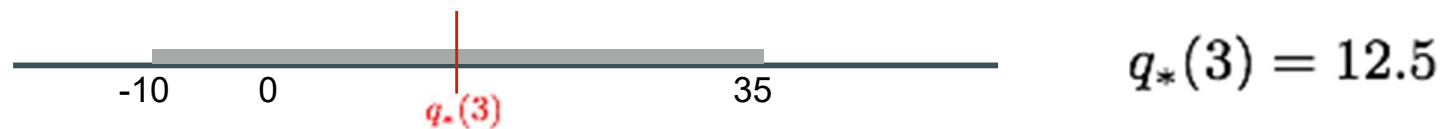
- Action: pull arm 4 — a third 0, a third 20, a third from $\{8, 9, \dots, 18\}$



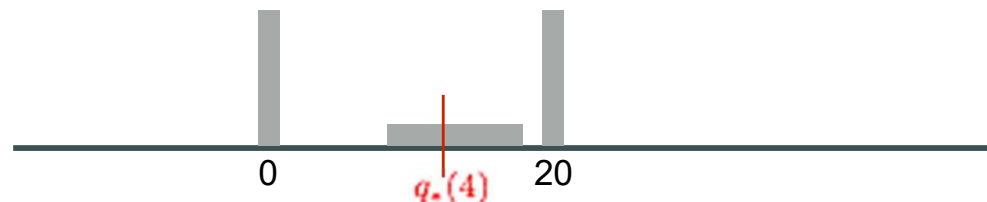
$$q_*(4) = \frac{1}{3} \times 0 + \frac{1}{3} \times 20 + \frac{1}{3} \times 13$$

Assume that there are $k=4$ bandits

- Action: pull arm 1 — Reward is always 8
 - value of arm 1 is $q_*(1) = 8$
- Action: pull arm 2 — 88% chance of 0 and 12% chance of 100!
 - value of arm 2 is $q_*(2) = .88 \times 0 + .12 \times 100 = 12$
- Action: pull arm 3 — Randomly between -10 and 35, equiprobable



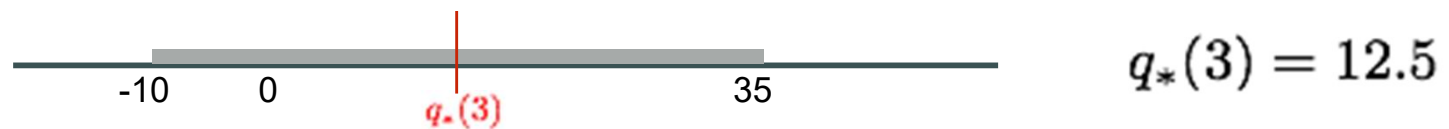
- Action: pull arm 4 — a third 0, a third 20, a third from $\{8, 9, \dots, 18\}$



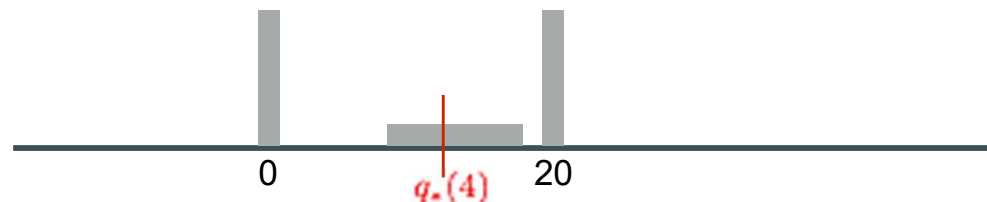
$$q_*(4) = \frac{1}{3} \times 0 + \frac{1}{3} \times 20 + \frac{1}{3} \times 13 = 0 + \frac{20}{3} + \frac{13}{3}$$

Assume that there are $k=4$ bandits

- Action: pull arm 1 — Reward is always 8
 - value of arm 1 is $q_*(1) = 8$
- Action: pull arm 2 — 88% chance of 0 and 12% chance of 100!
 - value of arm 2 is $q_*(2) = .88 \times 0 + .12 \times 100 = 12$
- Action: pull arm 3 — Randomly between -10 and 35, equiprobable

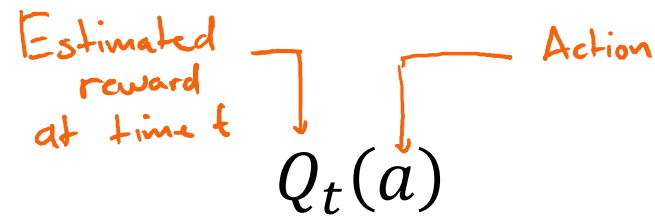


- Action: pull arm 4 — a third 0, a third 20, a third from $\{8,9,\dots, 18\}$



$$q_*(4) = \frac{1}{3} \times 0 + \frac{1}{3} \times 20 + \frac{1}{3} \times 13 = 0 + \frac{20}{3} + \frac{13}{3} = \frac{33}{3} = 11$$

We don't know $q_*(a)$, but we want to estimate it. We call our estimate



The k -armed Bandit Problem

- On each of a sequence of *time steps*, $t=1, 2, 3, \dots$, you choose an action A_t from k possibilities, and receive a real-valued *reward* R_t
- The reward depends only on the action taken; it is identically, independently distributed (i.i.d.):

$$q_*(a) \doteq \mathbb{E}[R_t | A_t = a], \quad \forall a \in \{1, \dots, k\} \quad \text{true values}$$

- These true values are *unknown*. The distribution is unknown.
- Nevertheless, you must maximize your total reward
- You must both try actions to learn their values (*explore*), and prefer those that appear best (*exploit*)

Bounded Reward Assumption

A common assumption we will make is that rewards are in a bounded interval $[-R_{max}, R_{max}]$.

I.e., for each i , $\Pr(R(a_i) \in [-R_{max}, R_{max}]) = 1$.

Note that results are available for other types of assumptions, e.g. Gaussian distributions of rewards.

Multi-Armed Bandits: Application examples

- **Clinical Trials**

- Arms: possible treatments
- Arm pulls: application of treatment to individual
- Rewards: outcome of treatment
- Objective: maximize cumulative reward = maximize benefit to trial population (or find best treatment quickly)

Multi-Armed Bandits: Application examples

- **Clinical Trials**

- Arms: possible treatments
- Arm pulls: application of treatment to individual
- Rewards: outcome of treatment
- Objective: maximize cumulative reward = maximize benefit to trial population (or find best treatment quickly)

- **Online Advertising**

- Arms: different ads/ad-types for a webpage
- Arm pulls: displaying an ad upon a page access
- Rewards: click through
- Objective: maximize cumulative reward = maximize clicks (or find best add quickly)

Assuming that you have k bandits, how would you maximize the rewards?



Remember: bandits pay out a reward chosen from probability distributions. Each bandit has its own probability distribution, and you have no idea what they are.

UniformBandit Algorithm

1. Pull each arm a fixed number (w) times
2. Return arm with best average reward



$r_{11} \ r_{12} \ \dots \ r_{1w}$



$r_{21} \ r_{22} \ \dots \ r_{2w}$

...



$r_{k1} \ r_{k2} \ \dots \ r_{kw}$

Imagine that we have to
maximize the reward over a total
of 1000 arm pulls.

What should we do?

The Exploration/Exploitation Dilemma

- Suppose you form estimates

$$Q_t(a) \approx q_*(a), \quad \forall a \quad \text{action-value estimates}$$

- Define the *greedy action* at time t as

$$A_t^* \doteq \arg \max_a Q_t(a)$$

- If $A_t = A_t^*$ then you are *exploiting*
If $A_t \neq A_t^*$ then you are *exploring*
- You can't do both at the same time.

Action-Value Methods

- Methods that learn **action-value estimates** and nothing else
- For example, estimate action values as *sample averages*:

$$Q_t(a) \doteq \frac{\text{sum of rewards when } a \text{ taken prior to } t}{\text{number of times } a \text{ taken prior to } t} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbb{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbb{1}_{A_i=a}}$$

- The sample-average estimates converge to the true values
If the action is taken an infinite number of times

$$\lim_{N_t(a) \rightarrow \infty} Q_t(a) = q_*(a)$$

The number of times action a
has been taken by time t



ϵ -Greedy Action Selection

- In **greedy action selection**, you always exploit
- In **ϵ -greedy**, you are usually greedy, but with probability ϵ you instead pick an action at random (possibly the greedy action again)
- This is perhaps the simplest way to balance exploration and exploitation

Assume that an action has been taken $n - 1$ times already. Our estimated reward of taking the action an n 'th time is then:

$$Q_n \doteq \frac{R_1 + R_2 + \cdots + R_{n-1}}{n - 1}$$

Not computationally
efficient



$$\begin{aligned} Q_{n+1} &= \frac{1}{n} \sum_{i=1}^n R_i \\ &= \frac{1}{n} \left(R_n + \sum_{i=1}^{n-1} R_i \right) \\ &= \frac{1}{n} \left(R_n + (n-1) \frac{1}{n-1} \sum_{i=1}^{n-1} R_i \right) \\ &= \frac{1}{n} (R_n + (n-1)Q_n) \\ &= \frac{1}{n} (R_n + nQ_n - Q_n) \\ &= Q_n + \frac{1}{n} [R_n - Q_n], \end{aligned}$$

A simple bandit algorithm

Initialize, for $a = 1$ to k :

$$Q(a) \leftarrow 0$$

$$N(a) \leftarrow 0$$

Repeat forever:

$$A \leftarrow \begin{cases} \arg \max_a Q(a) & \text{with probability } 1 - \varepsilon \\ \text{a random action} & \text{with probability } \varepsilon \end{cases} \quad (\text{breaking ties randomly})$$

$$R \leftarrow \text{bandit}(A)$$

$$N(A) \leftarrow N(A) + 1$$

$$Q(A) \leftarrow Q(A) + \frac{1}{N(A)} [R - Q(A)]$$

Averaging as the learning rule

This is a standard form for learning/update rules:

$$NewEstimate \leftarrow OldEstimate + StepSize \underbrace{[Target - OldEstimate]}_{\text{Error}}$$

Potentially noisy

Learning rate α

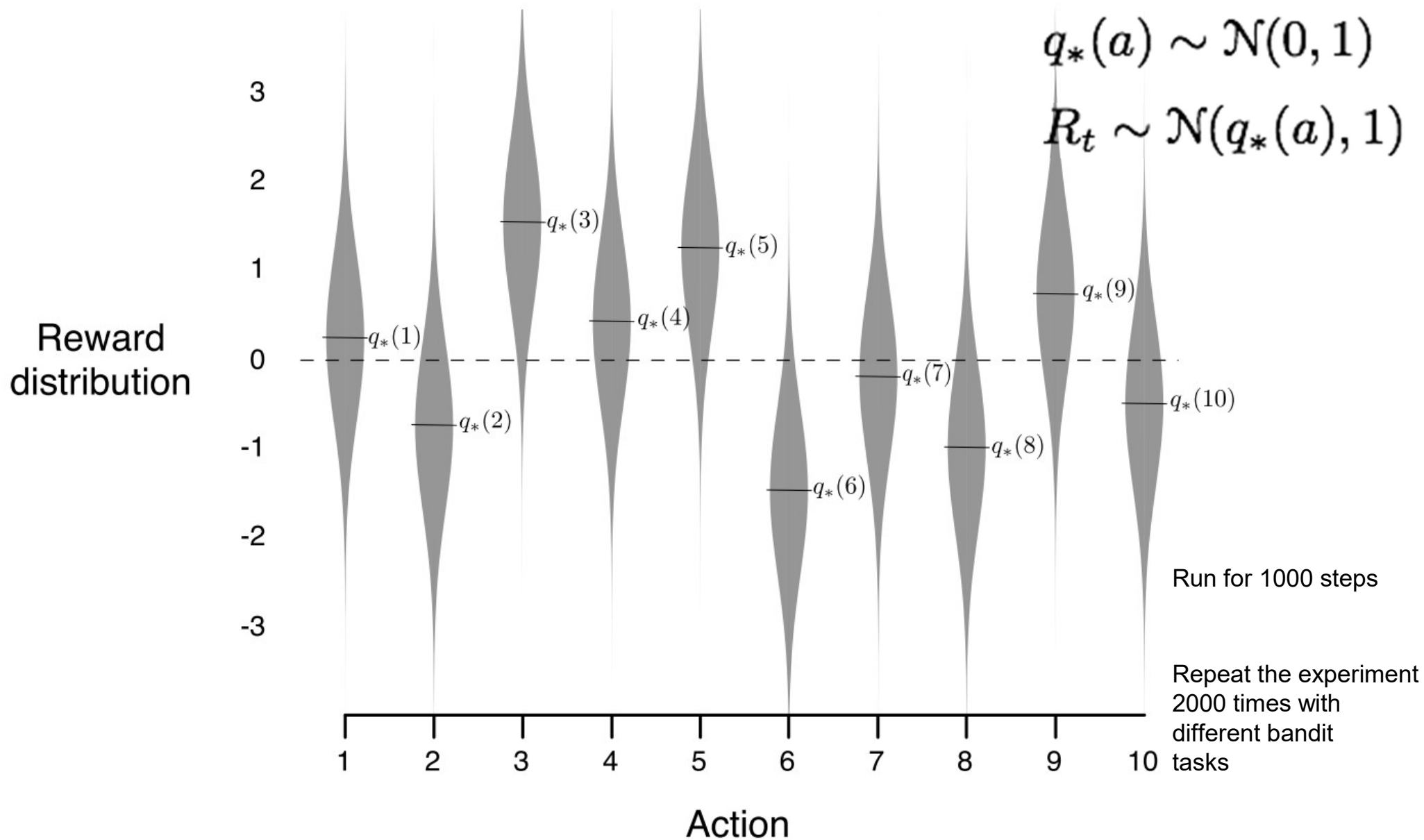
The diagram shows the equation $NewEstimate \leftarrow OldEstimate + StepSize [Target - OldEstimate]$. A handwritten orange arrow points from the text 'Potentially noisy' to the 'Target' term. Another handwritten orange arrow points from the text 'Learning rate α ' to the 'StepSize' term. A third handwritten orange bracket is drawn under the term '[Target - OldEstimate]', with the word 'Error' written below it.

Note, that the “*Target*” is presumed to indicate a desirable direction in which to move (may be noisy). The “*Target*” here is the n ’th reward.

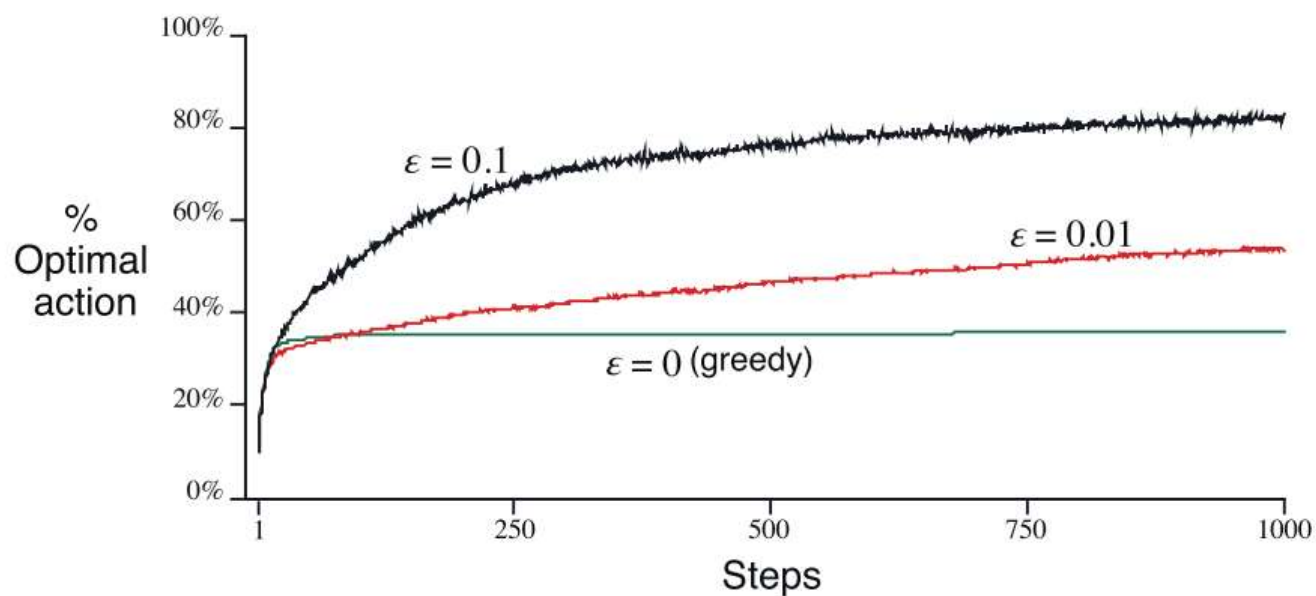
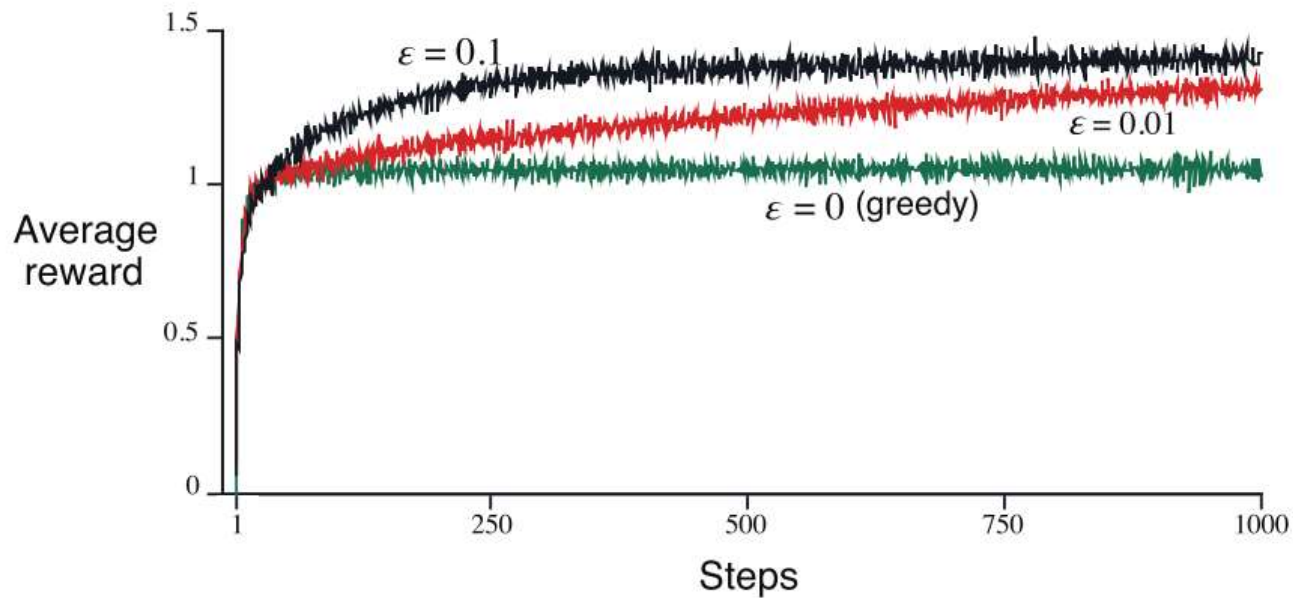
How well does ε -greedy action selection work?

Let's test for different values of ε .

The 10-armed Testbed



ϵ -Greedy Methods on the 10-Armed Testbed



Tracking a Non-stationary Problem

- Suppose the true action values **change slowly over time** - then we say that the problem is *non-stationary*
- In this case, sample averages are not a good idea (Why?)
- Better is an “*exponential, recency-weighted average*”:

$$Q_{n+1} \doteq Q_n + \alpha [R_n - Q_n]$$

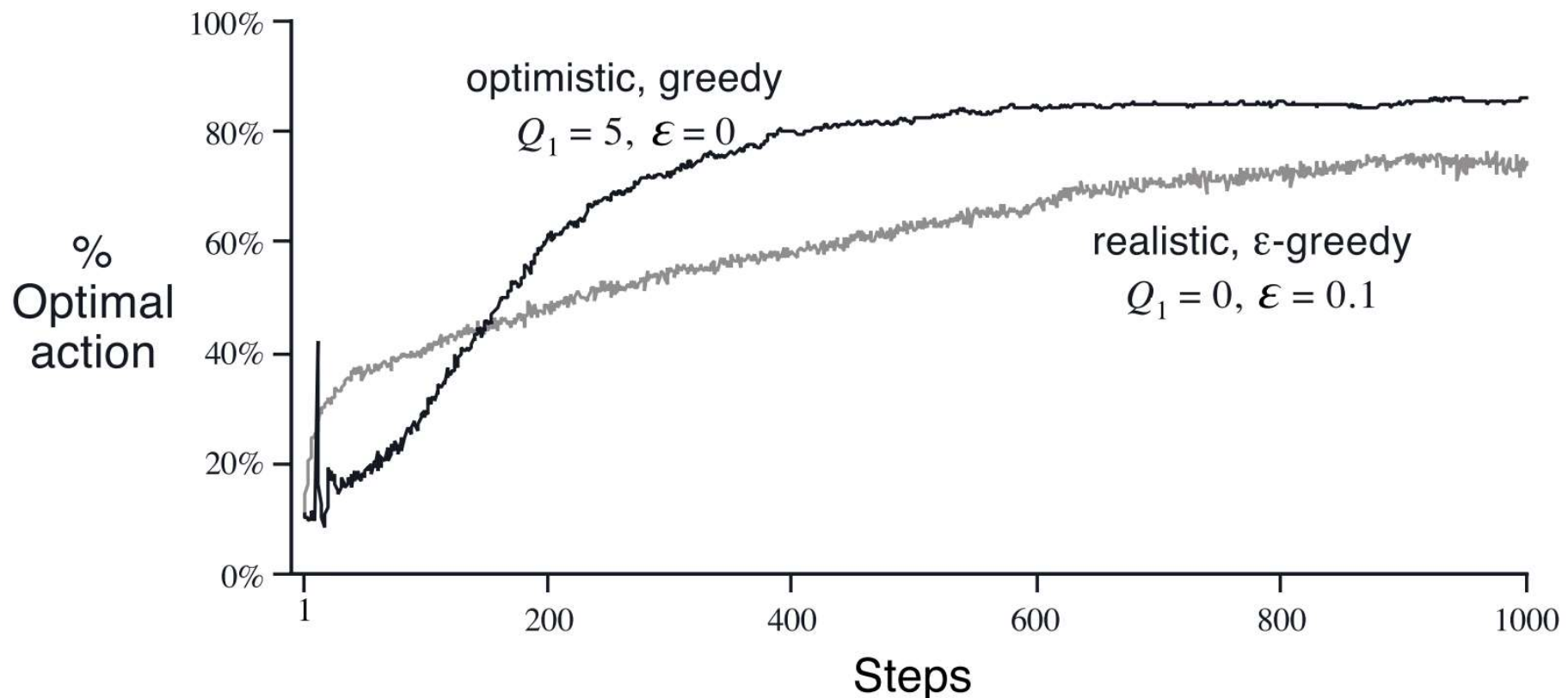
Where α , the step-size or learning rate, is constant.

- What is the effect if setting $\alpha = 1$?
- What is the effect if setting $\alpha = 0$?
- Which value should we assign to α ?

We will still focus on stationary problems, but remove the initial bias over time by setting α to a constant value.

Optimistic Initial Values

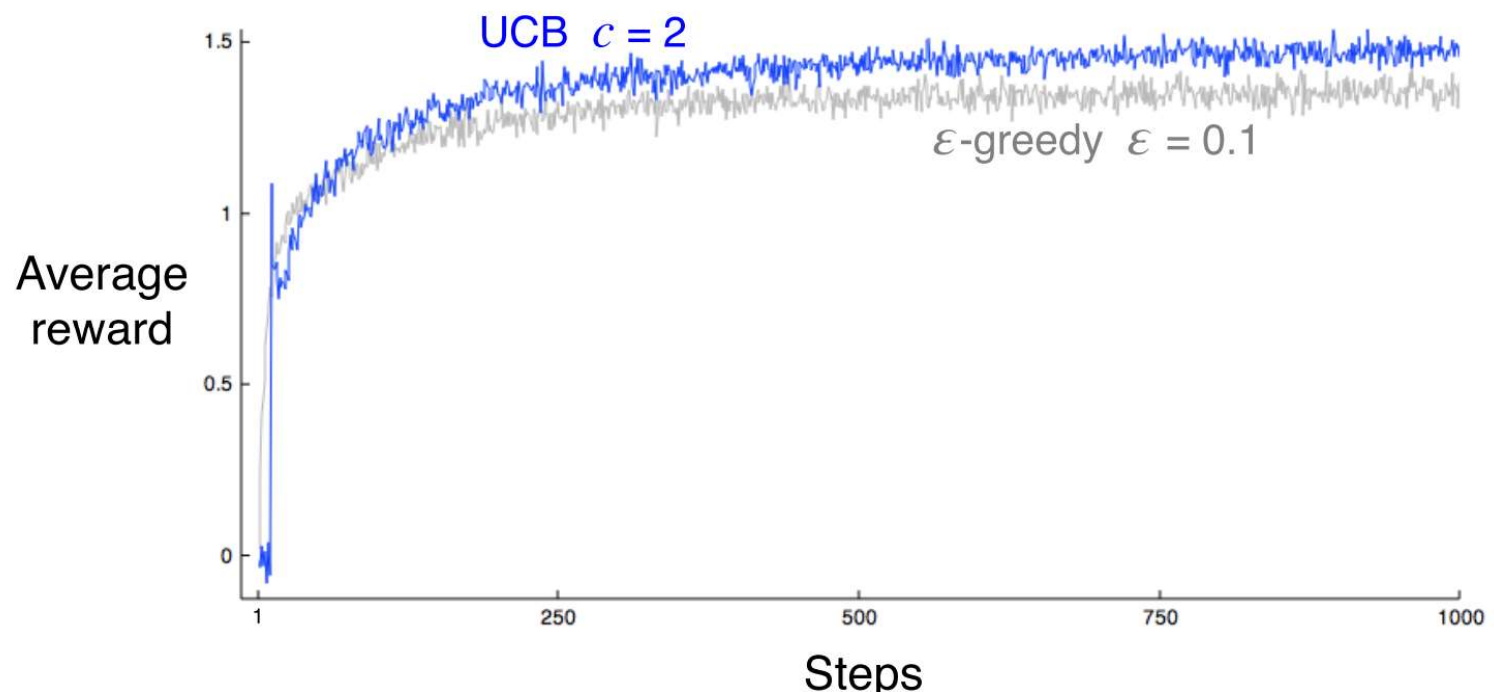
- All methods so far depend on $Q_1(a)$, i.e., they are biased. So far, we have used $Q_1(a) = 0$.
- Suppose we initialize the action values *optimistically* ($Q_1(a) = 5$), on the 10-armed testbed (with $\alpha = 0.1$)



Upper Confidence Bound (UCB) action selection

- A clever way of reducing exploration over time
- Estimate an upper bound on the true action values
- Select the action with the largest (estimated) upper bound

$$A_t \doteq \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$



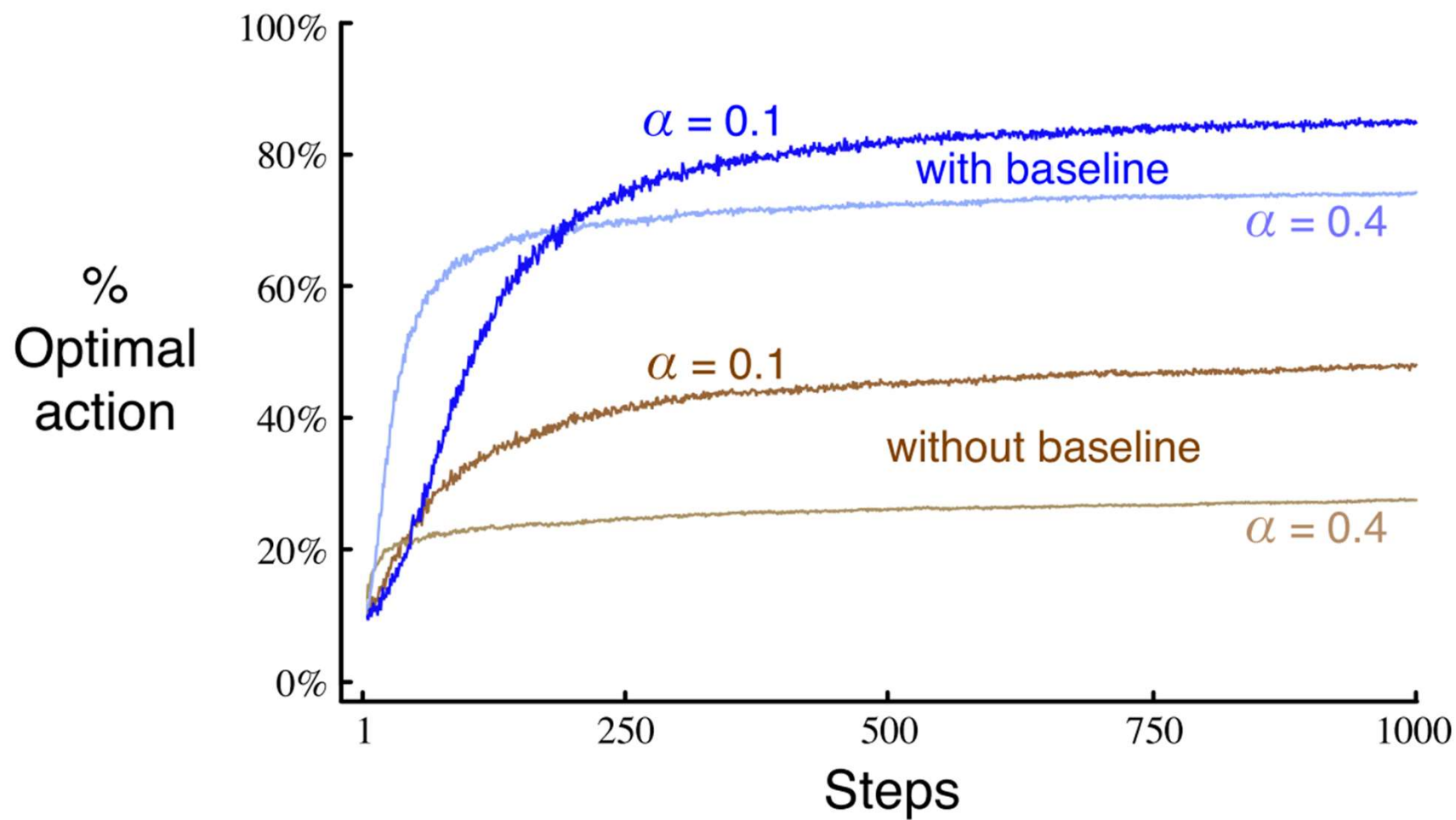
Gradient Bandit Algorithm

- We don't try to estimate q_* or rewards, we just keep track of our preference (or probability) for choosing an action:

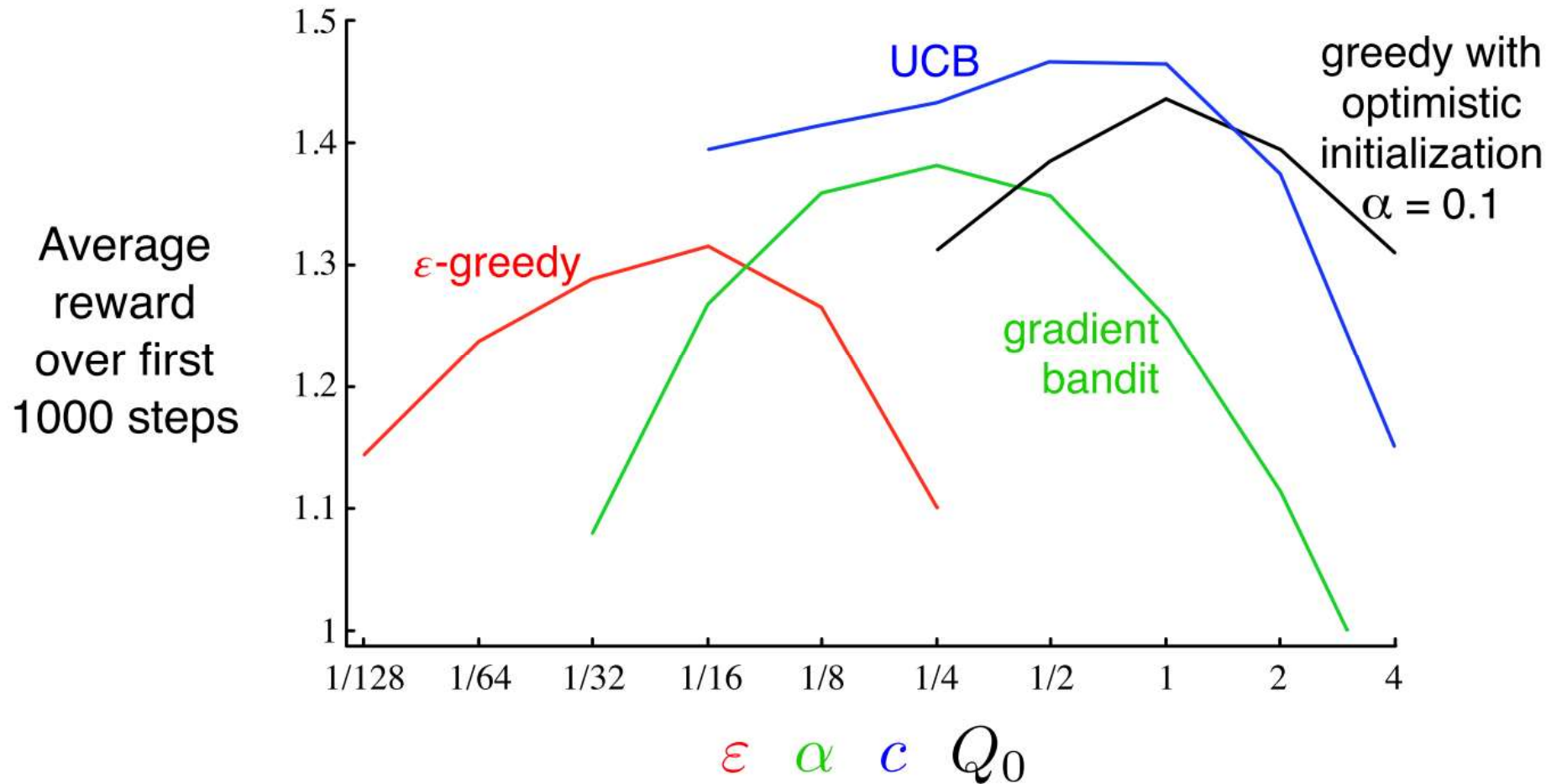
$$\Pr\{A_t = a\} \doteq \frac{e^{H_t(a)}}{\sum_{b=1}^k e^{H_t(b)}} \doteq \pi_t(a)$$

- We learn preferences according to:

$$\begin{aligned} H_{t+1}(A_t) &\doteq H_t(A_t) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(A_t)), & \text{and} \\ H_{t+1}(a) &\doteq H_t(a) - \alpha(R_t - \bar{R}_t)\pi_t(a), & \text{for all } a \neq A_t, \end{aligned}$$



Results



Summary 1/2

- *Reinforcement learning*: an agent learns through interactions with an environment.
- *k*-armed bandit problem: The true distribution of rewards is not known a priori, but must be learned.



$$q_*(a) \doteq \mathbb{E}[R_t \mid A_t = a]$$

$$Q_t(a) \approx q_*(a), \quad \forall a$$

Summary 2/2

We discussed:

- UniformBandit
- Exploration vs. exploitation tradeoff
- ε -Greedy
- How to reduce the bias of initial estimates $Q_1(a)$, namely a fixed step-size α .
- Optimistic initialization ($Q_1(a) = 5$)
- Upper confidence bound: $A_t \doteq \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$
- Gradient bandit algorithm: preferences are used instead of estimates of q_*